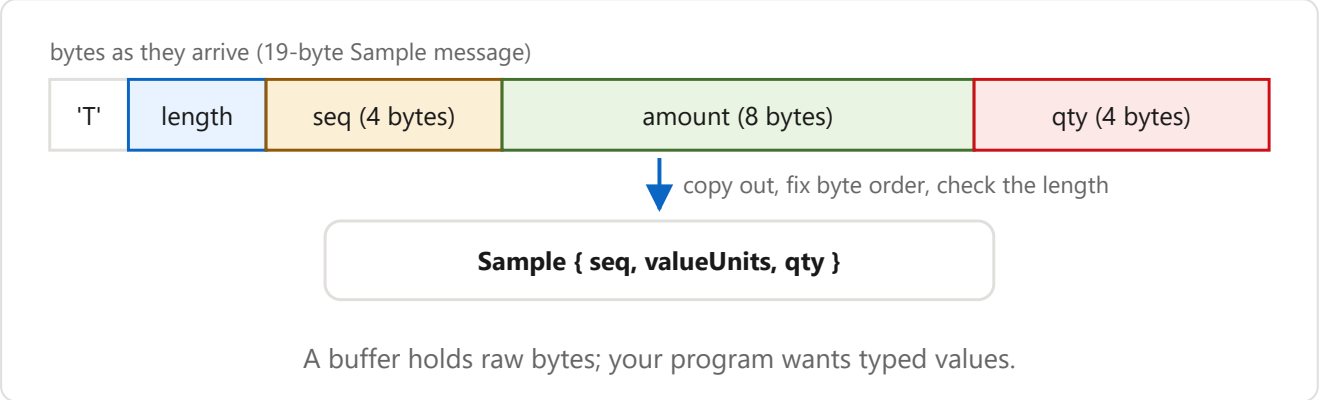
Binary Data and Parsing

How a program turns bytes from the network into values: bit operations, struct layout, byte order, safe reads, zero-copy parsing, amounts and sequence numbers. Compiled examples are in `examples/lessons/11-wire-formats`.

From bytes to values



A network message arrives as bytes. The job: turn them into values **quickly, correctly and without undefined behavior**.

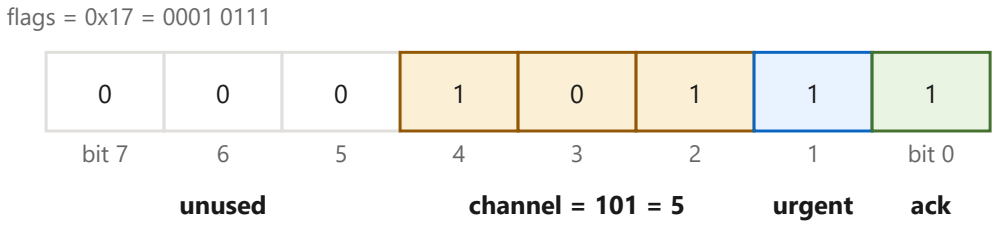
- **Never cast the buffer to a struct pointer.** Copy fields out instead.
- **Check the length before every read.** A truncated or corrupt packet must be rejected, not trusted.
- **Convert byte order explicitly.** The network and your CPU may disagree.

THE TEMPTING WRONG WAY

```
auto* t = reinterpret_cast<const Sample*>(buf);   // reads fields straight out of the buffer
std::cout << t->seq;
```

That one line hides four problems: **aliasing** (undefined behavior), **alignment** (the buffer address may not suit a `Sample`), **padding** (the struct layout may not match the wire) and **byte order**. The next sections take them one by one.

Integers and bit manipulation



Read the channel: (flags >> 2) & 0x7 shift the field down, then mask its width.

Protocols pack several fields into one byte; bit operations unpack them.

OPERATION	EXPRESSION
set bit n	`x | (1u << n)`
clear bit n	`x & ~(1u << n)`
toggle bit n	`x ^ (1u << n)`
test bit n	`(x >> n) & 1u`
power of two?	`x != 0 && (x & (x - 1)) == 0`
count set bits	`std::popcount(x)` (C++20), or loop `x &= x - 1` (clears the lowest set bit each time)

```cpp
constexpr std::uint8_t kAck = 1u << 0;
constexpr std::uint8_t kUrgent = 1u << 1;
constexpr unsigned channelOf(std::uint8_t f) { return (f >> 2) & 0x7u; }

std::uint8_t flags = kAck | kUrgent | (5u << 2);      // 0x17
bool ack = flags & kAck;                             // true
unsigned channel = channelOf(flags);                  // 5
```

Rules that bite

- Use **fixed-width types** (`std::uint8_t`, `std::uint32_t`, `std::int64_t`) for anything on the wire; `int` and `long` change size between platforms (`long` is 32-bit on Windows).
- **Unsigned overflow wraps** (well defined); **signed overflow is undefined behavior**.
- Shifting by the type's width or more is undefined: write `1u << n` with `n < 32`. Do bit work on **unsigned** types.
- Small types are promoted to `int` in arithmetic. A plain `char` may be signed, so widening `char(0xFF)` to `int` gives **-1**: read bytes as `unsigned char` or `std::uint8_t`.

▼ PREDICT **Promotion and wrap-around**

```cpp
std::uint8_t a = 200, b = 100;
std::cout << (a + b) << ' ' << int(std::uint8_t(a + b));
```

```
300 44
```

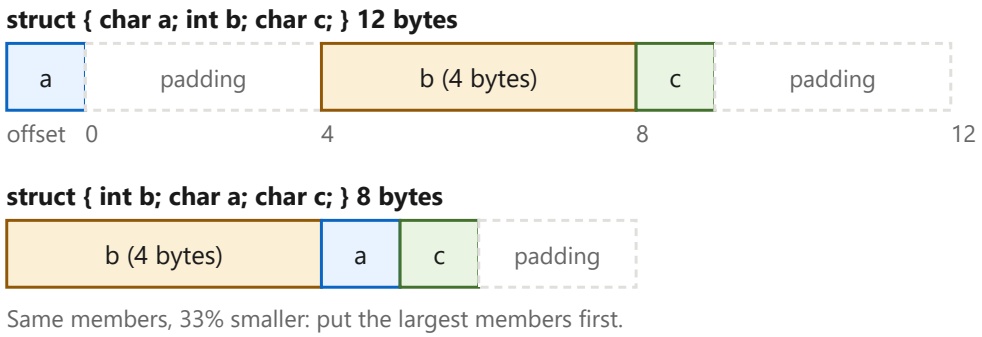
`a + b` is computed as an `int` (both promote), so it is **300**. Storing it back into a `uint8_t` keeps only the low 8 bits: 300 mod 256 = **44**.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 4.8 The bitwise operators (PDF pages 181 to 184)
- 17.2 The bitset type (PDF pages 752 to 756)

Struct layout, alignment and padding



Same members, 33% smaller: put the largest members first.

The compiler inserts padding so every member sits at a multiple of its alignment.

Each type has an **alignment** (`alignof`): an `int` must start at a multiple of 4, a `double` or `int64_t` at a multiple of 8. The compiler pads between members to guarantee that, and pads the end so that arrays of the struct stay aligned. So `sizeof` is often bigger than the sum of the members.

Step through the layout:

STEP THROUGH **Where the compiler puts each member**

```
1  struct Careless {
2      char a;
3      int  b;
4      char c;
5  };
6  sizeof(Careless)
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 2 `char a;`	**a** goes first, at offset 0. It takes 1 byte.	**`a = 1 byte`**
2	line 3 `int  b;`	An `int` must start at a multiple of 4, so the compiler leaves **3 padding bytes** and places **b at offset 4**.	`a = 1 byte` | **`padding = unused`** | **`b = 4 bytes`**
3	line 4 `char c;`	**c** needs no alignment, so it follows b at offset 8. Members so far end at byte 9.	`a = 1 byte` | `padding = unused` | `b = 4 bytes` | **`c = 1 byte`**
4	line 5 `};`	The whole struct is padded up to a multiple of its alignment (4), so an **array** of them keeps every `b` aligned. Total: **12 bytes**, 5 of them padding.	`a = 1 byte` | `padding = unused` | `b = 4 bytes` | `c = 1 byte` | **`padding = unused`**
5	line 6 `sizeof(Careless)`	The program prints **12**. Reorder the members as `int b; char a; char c;` and the same data needs only **8 bytes** (shown): put the largest members first. prints: `12\n`	**`b = 4 bytes`** | **`a = 1 byte`** | **`c = 1 byte`** | **`padding = unused`**

```
struct Careless { char a; int b; char c; };         // 12 bytes
struct Ordered  { int b; char a; char c; };         //  8 bytes

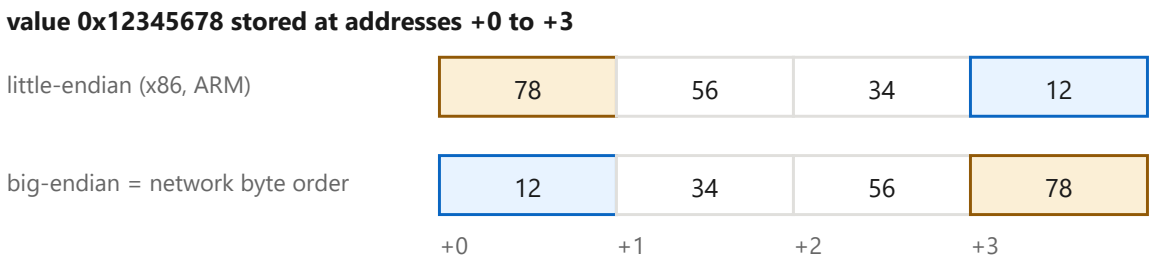
static_assert(sizeof(Careless) == 12 && alignof(Careless) == 4);
static_assert(offsetof(Careless, b) == 4);          // offsetof gives the
byte offset of a member
```

Packing, and why not to rely on it

```
#pragma pack(push, 1)
struct Packed { char a; int b; char c; };           // 6 bytes, no padding:
matches a wire layout
#pragma pack(pop)
```

> **PACKING IS NOT FREE** A packed member may be **unaligned**: slower on some CPUs, a fault on others, and taking a reference or pointer to it is unsafe. Do not read the wire into a packed struct and use it directly. Read the bytes with `memcpy` (next sections) and keep ordinary structs inside your program. `static_assert(sizeof( ... ))` is a cheap guard against accidental layout changes.

Endianness



Little-endian stores the least significant byte first, big-endian the most significant first.

Same value, different byte order in memory.

- **Endianness** is the order of a multi-byte value's bytes in memory. Network protocols traditionally use **big-endian** ("network byte order"); most desktop and server CPUs are little-endian. Protocols differ, so read each protocol's specification.
- **Convert at the boundary**: when bytes enter the program, turn them into native values, and stay native inside.
- Tools: `ntohl` / `htonl` (sockets), `std::endian::native` (C++20) to test the CPU, `std::byteswap` (C++23). The portable way that works everywhere is to **assemble the value from bytes with shifts**:

```cpp
constexpr std::uint32_t readBe32(const unsigned char* p) {
    return (std::uint32_t{p[0]} << 24) | (std::uint32_t{p[1]} << 16)
         | (std::uint32_t{p[2]} <<  8) |  std::uint32_t{p[3]};
}
const unsigned char wire[] = {0x12, 0x34, 0x56, 0x78};
readBe32(wire);                                        // 0x12345678 on ANY CPU
```

Widen each byte to `uint32_t` *before* shifting, otherwise it promotes to a signed `int` and shifting a large value into the top bit is a trap. The same code works on little- and big-endian machines, and compilers turn it into a single load plus a byte swap.

Reading bytes safely: aliasing and memcpy

Strict aliasing: you may access an object only through a pointer or reference of its own type (or a few related ones). Reading a `uint32_t` out of memory that holds something else is undefined behavior, and the optimizer is allowed to assume it never happens.

TECHNIQUE	DEFINED?	NOTES
`*reinterpret_cast<const uint32_t*>(buf + 1)`	**no**	aliasing violation, and `buf + 1` is probably misaligned
`std::memcpy(&v, buf + 1, sizeof v)`	yes	compilers turn it into a plain load; no alignment requirement on `buf`
`std::bit_cast<To>(from)` (C++20)	yes	same size, trivially copyable types, e.g. float to its bits
assemble with shifts (`readBe32`)	yes	also fixes byte order
write through a `union` and read another member	**no** in C++	works as a GCC extension, but is not portable C++
view an object as `unsigned char`, `char` or `std::byte`	yes	the one exception: these may alias anything (but not the reverse)

```cpp
std::uint32_t v;
std::memcpy(&v, buf + 1, sizeof v);                 // native byte order: add
a swap if the wire is big-endian

std::bit_cast<std::uint32_t>(1.0f);                 // 0x3f800000: the bits
of a float
```

KEY POINT "Why is casting the receive buffer to a struct pointer wrong?" Aliasing (UB), alignment, padding that may not match the wire, and byte order. "What do you do instead?" `memcpy` or shifts per field, after checking the length.

Zero-copy parsing

A **zero-copy** parser never copies the message: it keeps views (`std::string_view`, `std::span`) into the receive buffer and a cursor that advances. No allocation, no copying, no exceptions on the hot path.

```cpp
struct Field { int tag; std::string_view value; };

// "1=D|2=north|3=12345|"  →  (1,"D")  (2,"north")  (3,"12345")
std::optional<Field> nextField(std::string_view& rest) {
    auto bar = rest.find('|');
    if (bar == std::string_view::npos) return std::nullopt;       //
incomplete: reject
    std::string_view item = rest.substr(0, bar);
    rest.remove_prefix(bar + 1);                                   // advance
the cursor
    auto eq = item.find('=');
    if (eq == std::string_view::npos) return std::nullopt;
    Field f{};
    auto r = std::from_chars(item.data(), item.data() + eq, f.tag);
    if (r.ec ≠ std::errc{}) return std::nullopt;
    f.value = item.substr(eq + 1);                                 // a view:
no copy
    return f;
}
```

- `std::from_chars` (C++17) converts text to numbers with **no allocation, no exceptions and no locale**; it returns an error code and the pointer where it stopped. Compare with `stoi` (throws, needs a `std::string`) and `stringstream` (slow).
- **Validate everything** before trusting it: length fields, tags, digits. Corrupt input is normal; return `std::nullopt` or an error code instead of throwing.
- A view is valid only while the **buffer is alive and unchanged**. If the buffer is reused for the next packet, copy out what you need first.

Step through the parse of a binary message (all integers big-endian):

STEP THROUGH **Parsing a 19-byte Sample message**

```
1  if (len < 19) return nullopt;
2  type   = buf[0];
3  length = loadBe<uint16_t>(buf + 1);
4  seq    = loadBe<uint32_t>(buf + 3);
5  amount  = loadBe<uint64_t>(buf + 7);
6  qty    = loadBe<uint32_t>(buf + 15);
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `if (len < 19) return nullopt;`	**Check the length first.** All 19 bytes must be present, or any read below could run past the end of the buffer. A short packet is rejected, not trusted.	**type = 'T' | length = 00 13 | seq = 00 00 30 39 | amount = 00 00 00 00 00 0F 73 14 | qty = 00 00 01 2C**
2	line 2 `type = buf[0];`	One byte: no byte-order issue. A wrong type means this is not a Sample, so reject it.	type = 'T' | length = 00 13 | seq = 00 00 30 39 | amount = 00 00 00 00 00 0F 73 14 | qty = 00 00 01 2C | **p = buf + 0** | **msgType = 'T'**
3	line 3 `length = loadBe<uint16_t>(buf + 1);`	`loadBe<uint16_t>` assembles the two bytes `00 13` as big-endian: **19**. It must equal the message size, otherwise the packet is corrupt.	type = 'T' | length = 00 13 | seq = 00 00 30 39 | amount = 00 00 00 00 00 0F 73 14 | qty = 00 00 01 2C | **p = buf + 1** | **len = 19**
4	line 4 `seq = loadBe<uint32_t>(buf + 3);`	Four bytes `00 00 30 39` become **12345** (0x3039). Each field is copied out; the buffer is never reinterpreted as a struct.	type = 'T' | length = 00 13 | seq = 00 00 30 39 | amount = 00 00 00 00 00 0F 73 14 | qty = 00 00 01 2C | **p = buf + 3** | **seqNo = 12345**
5	line 5 `amount = loadBe<uint64_t>(buf + 7);`	Eight bytes become **1012500** units, which is 101.2500 at 4 decimals. An integer, so no floating-point error.	type = 'T' | length = 00 13 | seq = 00 00 30 39 | amount = 00 00 00 00 00 0F 73 14 | qty = 00 00 01 2C | **p = buf + 7** | **valueUnits = 1012500**
6	line 6 `qty = loadBe<uint32_t>(buf + 15);`	**qty = 300**. The parse used 19 bytes, did not allocate, and could not read past the end because the length was checked at the start.	type = 'T' | length = 00 13 | seq = 00 00 30 39 | amount = 00 00 00 00 00 0F 73 14 | qty = 00 00 01 2C | **p = buf + 15** | **qtyValue = 300**

Money amounts: fixed-point versus double

```
0.1 + 0.2 == 0.3;       // false
0.1 + 0.2;              // 0.30000000000000004
```

`double` cannot represent most decimal fractions exactly, so sums and comparisons drift. For money and amounts use an **integer count of units**: 101.2500 with a unit of 0.0001 is `1012500`.

```
class Money {
    std::int64_t units_;                                    // 1 unit = 0.0001
public:
    static constexpr std::int64_t kScale = 10000;
    constexpr Money operator+(Money o) const { return Money(units_ +
o.units_); }
    constexpr auto operator<=>(const Money&) const = default;    // exact
comparison, no epsilon
    static std::optional<Money> parse(std::string_view s);      // "101.25" →
1012500, rejects a 5th decimal
};
// 0.1 + 0.2 == 0.3 is true here: 1000 + 2000 == 3000
```

- **Exact and deterministic**: the same inputs give the same bits on every machine, and equality needs no epsilon.
- **Parse from text directly** into units; never go through `double`.
- **Watch overflow**: `int64_t` holds about 9.2e18 units (9.2e14 amount units at 4 decimals). Multiplying amount by quantity can exceed that: check, or use a wider type (`__int128` in GCC and Clang).
- Decide the rounding rule (down, half-up, half-even) once, and apply it in one place.

Sequence numbers and gaps



Each message carries a sequence number; a jump means something was lost.

RECEIVED VERSUS EXPECTED	MEANING	TYPICAL ACTION
equal	in order	process, expect `seq + 1`
higher	**gap**: messages were lost	buffer new messages, request a retransmission or a snapshot
lower	duplicate or old	ignore

```
enum class SeqResult { InOrder, Duplicate, Gap };

SeqResult classify(std::uint32_t expected, std::uint32_t received) {
    auto diff = static_cast<std::int32_t>(received - expected);   // unsigned subtraction wraps, then read it as signed
    if (diff == 0) return SeqResult::InOrder;
    return diff < 0 ? SeqResult::Duplicate : SeqResult::Gap;
}
```

Wrap-around: a 32-bit counter returns to 0 after 4294967295. Comparing with `<` breaks at that point; the unsigned subtraction followed by a signed reinterpretation keeps working as long as the two values are within 2^{31} of each other. Expecting 4294967295 and receiving 0 is a **gap of one**, not a duplicate.

Cheat sheet

Answer aloud, then open.

<details>
<summary>Why not cast the receive buffer to a struct pointer?</summary>

Four problems at once: it violates strict aliasing (undefined behavior), the buffer address may not be aligned for the struct, struct padding may not match the wire layout, and the byte order may differ. Check the length, then copy each field out with `memcpy` or assemble it with shifts.
</details>

<details>
<summary>What is struct padding and how do you reduce it?</summary>

The compiler inserts unused bytes so each member sits at a multiple of its alignment, and pads the end so arrays stay aligned. `sizeof` can exceed the sum of the members. Order members from largest to smallest alignment; use `static_assert(sizeof( ... ))` to guard the size.
</details>

<details>
<summary>What does `#pragma pack(1)` do, and what does it cost?</summary>

It removes padding so the struct matches a wire layout. The cost: members may be unaligned, which is slower or faults on some CPUs, and references or pointers to them are unsafe. Prefer reading fields with `memcpy`.
</details>

<details>
<summary>What is endianness? Which does the network use?</summary>

The order of a multi-byte value's bytes in memory. Little-endian stores the least significant byte first (x86, ARM), big-endian the most significant first. Network byte order is big-endian, though a given protocol may differ, so check the spec. Convert at the boundary.
</details>

▼ How do you read a big-endian 32-bit value portably?

Assemble it from the bytes with shifts, widening each byte to `uint32_t` first: `(uint32_t{p[0]} << 24) | (uint32_t{p[1]} << 16) | (uint32_t{p[2]} << 8) | uint32_t{p[3]}`. It works on any CPU.

▼ What is strict aliasing?

The rule that an object may only be accessed through an lvalue of its own type (or a few related types); the compiler assumes this when optimizing. Exceptions: `char`, `unsigned char` and `std::byte` may alias anything. Use `memcpy` or `std::bit_cast` to reinterpret bytes.

▼ `memcpy`, `bit_cast` or a union for type punning?

`memcpy` and `std::bit_cast` (C++20) are well defined and compile to a plain load. Reading a different union member than was written is undefined behavior in C++, even though GCC allows it as an extension.

▼ How do you set, clear and test a bit, and check for a power of two?

Set `x | (1u << n)`; clear `x & ~(1u << n)`; test `(x >> n) & 1`. Power of two: `x ≠ 0 && (x & (x - 1)) == 0`. Use unsigned types and keep `n` below the width.

▼ Why should amounts not be `double`?

Binary floating point cannot represent most decimals exactly (`0.1 + 0.2 ≠ 0.3`), so sums and comparisons drift. Store an integer count of units (a scaled `int64_t`): exact, deterministic, and comparable with `==`. Parse text straight into units and watch for overflow when multiplying by quantity.

▼ Why `std::from_chars` in a parser?

It converts text to numbers without allocating, throwing or consulting the locale, and reports errors through a return value plus the position where it stopped. `stoi` throws and needs a `std::string`; `stringstream` is slow.

▼ How do you detect a gap in sequence numbers, and handle wrap-around?

Compare the received number with the expected one: equal is in order, higher is a gap (lost messages: buffer and request a retransmission or snapshot), lower is a duplicate. For wrap-around compute `static_cast<int32_t>(received - expected)` and look at its sign.